Vivekanand Education Society's Institute Of Technology (VESIT)
Department of Information Technology
Course: **Blockchain**

Mini Project

Cookbook / Operations Manual

TrustChain

Blockchain-Based Transparent Charity Donation
and Fund Tracking System Using Smart Contracts

Roll No.	Name
23	Janhavi Patil
29	Khushi Shukla
30	Khushi singh
34	Jai Udasi

Group No.: 10

Faculty In-Charge: Kajal Jewani

Repository: https://github.com/khushi-shukla-cz/blockchain-workbench/tree/main/Mini-Project

1. Overview

TrustChain is a blockchain-based charity donation and fund-tracking system. It is designed to make donation handling and campaign fund movements more transparent by using smart contracts for core on-chain operations. The project combines a Next.js web interface, a local blockchain development environment, wallet-based transaction signing, and a supporting backend API.

Core campaign and donation operations are handled by Solidity smart contracts. The backend provides supporting services such as campaign metadata storage, evidence-file hashing and verification, alert retrieval, PDF campaign reports, and health checks. The backend is supporting infrastructure and is not the transaction processor for on-chain donations.The current campaign-creation interface does not automatically synchronize newly created campaigns to PostgreSQL, as noted in the project README. On-chain campaign state and database-backed supporting metadata should therefore be treated as separate parts of the current implementation.

1.1 Main Features

1. Campaign creation and approval through the CampaignManager smart contract.
2. Donations, escrow handling, releases, and refunds through DonationEscrow.
3. Milestone and governance workflows through dedicated smart contracts.
4. NGO registration, audit records, and role-based access controls.
5. MetaMask wallet connection and transaction signing through ethers.js.

1.2 Core Stack

Layer	Tooling
Frontend	Next.js
Blockchain	Solidity, Hardhat, ethers.js
Local Network	Ganache (chain ID 1337; RPC port 7545)
Wallet	MetaMask
Backend	Node.js, Express
Database Support	PostgreSQL-oriented supporting features

2. Prerequisites

- Node.js and npm (use versions compatible with the package files).
- Git for cloning and managing the repository.
- Ganache for the local Ethereum-compatible blockchain.
- MetaMask browser extension for wallet connection.
- PostgreSQL if using backend features that require the database.

2.1 Install Dependencies

git clone https://github.com/khushi-shukla-cz/blockchain-workbench.git
cd blockchain-workbench/Mini-Project
npm install

Install dependencies in the blockchain, backend, and frontend directories using their package manifests:

cd blockchain && npm install
cd ../backend && npm install
cd ../frontend && npm install

3. Setup

3.1 Environment Setup

The repository includes .env.example files for the frontend, backend, and blockchain directories. Copy the relevant example files to local environment files and provide the values required by the local setup.

Do not commit populated .env files, private keys, or wallet recovery phrases.

3.2 Start Local Blockchain

Start Ganache and confirm that the configured local network is available. The project configuration uses chain ID 1337 and RPC port 7545.

Configure MetaMask to use the same RPC endpoint and chain ID.

3.3 Compile and Deploy Contracts

cd blockchain
npx hardhat compile
npx hardhat run scripts/deploy.js --network <configured-network>

Use the network name and deployment command defined in blockchain/hardhat.config.js and blockchain/package.json.

After deployment, ensure frontend/src/lib/deployments.json contains the correct contract addresses for the selected network.

3.4 Start Backend and Frontend

Start the backend:

```
cd backend
npm run dev
```

Start the frontend:

```
cd frontend
npm run dev
```

Use the exact scripts defined in the respective package.json files if the script names differ.

4. Configuration

4.1 Ganache and MetaMask

Configure MetaMask with the Ganache RPC endpoint and chain ID 1337.

Use funded Ganache test accounts for local demonstrations only. The frontend blockchain integration is located in:

frontend/src/lib/chain.ts

4.2 Contract Addresses

The frontend reads deployed contract references from:

frontend/src/lib/deployments.json

If the frontend cannot interact with a contract, verify that the address belongs to the current Ganache instance and that the selected MetaMask network is correct.

4.3 Backend Configuration

Backend environment values are configured using:

backend/.env.example

The backend entry point is:

backend/src/server.js

Database connection details and API settings depend on the local environment configuration.

5. Running the System

5.1 Start the Local Development Environment

Start Ganache, deploy the contracts, then launch the backend and frontend services.

Keep the terminal processes running while using the application.

5.2 Access the Web Application

Open the local URL printed by the Next.js development server (commonly http://localhost:3000, subject to the frontend configuration).

Connect MetaMask and select the configured Ganache network.

5.3 Typical User Workflow

1. Connect a wallet through MetaMask.
2. Register an NGO where the workflow requires registration.
3. Create a campaign and complete the required approval process.
4. Donate to a campaign using a funded local test account.
5. Demonstrate escrow, milestone, release, or refund operations as supported by campaign state.
6. Review audit-related information and available backend reports or evidence-verification features.

6. Backend API and Supporting Features

The Node.js / Express backend provides supporting functionality for:

- Campaign metadata.
- Evidence-file hashing and verification.
- Alert retrieval.
- PDF campaign reports.

- Health checks.

These services complement the smart contracts; donation transactions and escrow state are handled on-chain.

7. Testing

The project includes a Hardhat test suite at:

blockchain/test/TrustChain.test.js

Run the tests from the blockchain directory using the scripts defined in blockchain/package.json.

cd blockchain
npx hardhat test

The following areas should be checked during validation. This list is a test checklist and does not represent claimed test results.

Test Area	**Expected Verification**
Compilation	Solidity contracts compile successfully.
Deployment	Deployment script deploys contracts and returns addresses.
Wallet Connection	MetaMask connects to the configured local network.
Campaign Flow	Campaign creation and approval enforce the configured permissions.
Donation Flow	Donation transaction updates the on-chain campaign / escrow state.
Escrow	Release and refund operations enforce contract conditions.
Milestones / Voting	Milestone and governance actions follow contract rules.
Backend	Health, alerts, evidence, and report endpoints respond when configured.

8. Viewing & Monitoring

8.1 Backend Health

The backend exposes health-check functionality. The exact route and port should be confirmed in backend/src/server.js and the active environment configuration.

8.2 Evidence Verification

The backend includes evidence-file hashing and verification support. Use the API route and request format implemented in backend/src/server.js.

8.3 Campaign Reports and Alerts

The backend includes alert retrieval and PDF campaign-report functionality. These are supporting services and should be interpreted alongside the corresponding on-chain campaign state.

9. Smart Contracts

Seven Solidity contracts are present under blockchain/contracts/:

Contract	**Purpose**
AccessController	Access-control functionality and permission management.
NGORegistry	NGO registration.
CampaignManager	Campaign creation and approval.
DonationEscrow	Donations, escrow, releases, and refunds.
MilestoneManager	Milestone-related campaign functionality.
GovernanceVoting	Governance and voting functionality.
AuditRegistry	Audit-related records.

10. Privacy, Trust & Security Considerations

Risk / Concern	**Implementation Consideration**
Unauthorized contract actions	Smart-contract access controls and permission checks should restrict privileged operations.
Incorrect donation handling	Donation and escrow state changes are performed through the DonationEscrow contract.
Campaign data mismatch	Campaign creation is not currently automatically synchronized to PostgreSQL; compare supporting metadata with on-chain state.
Evidence tampering	Backend evidence-file hashing and verification can be used to check file integrity.
Incorrect network or contract address	Confirm MetaMask network and frontend deployment addresses before submitting transactions.
Credential exposure	Do not commit private keys, wallet recovery phrases, or populated environment files.
Frontend-only validation	Do not rely on UI restrictions alone; contract-level checks are essential.

11 Services & Ports

Service	**Port / Setting**	**Package / Location**	**Role**
Ganache RPC	7545	Local development network	Local blockchain RPC endpoint
Frontend	Usually 3000	frontend	Next.js user interface
Backend	Configured in env/source	backend/src/server.js	Express API and supporting services
PostgreSQL	Configured in env	Backend configuration	Supporting metadata storage, where enabled

12. Troubleshooting

Symptom	Fix / Check
MetaMask cannot connect	Confirm Ganache is running and MetaMask uses the configured RPC endpoint and chain ID 1337.
Transaction is rejected	Check account balance, selected network, contract address, permissions, and contract preconditions.
Frontend cannot locate contracts	Verify deployment completed and deployments.json contains addresses for the selected network.
Hardhat compile or test fails	Run commands from blockchain and check dependencies and Hardhat configuration.
Backend fails to start	Check backend dependencies, environment variables, port configuration, and database availability if required.
Campaign data differs between UI and database	New campaign creation is not automatically synchronized to PostgreSQL in the current implementation.
Report / evidence operation fails	Check backend logs, endpoint configuration, file access, and database/storage dependencies.

13. Repository Map

The project is organized into separate frontend, backend, and blockchain components. The principal source files are:

```
TrustChain/
  README.md
  QUICK_START_MAC.md
  demo-guide.md
  setup.js
  start.js
  docker-compose.yml

  backend/
    src/server.js

  blockchain/
    contracts/
      AccessController.sol
```

```
    AuditRegistry.sol
    CampaignManager.sol
    DonationEscrow.sol
    GovernanceVoting.sol
    MilestoneManager.sol
    NGORegistry.sol

  scripts/deploy.js
  test/TrustChain.test.js
  hardhat.config.js

frontend/
  src/app/
    page.tsx
    layout.tsx
    globals.css

  src/lib/
    chain.ts
    deployments.json
```

14. References

Repository:
https://github.com/khushi-shukla-cz/blockchain-workbench/tree/main/Mini-Project